

Transforming the COMISET Windows Event Dataset into OCEL Logs for Process Mining and Predictive Monitoring

TL;DR

- **Use OCEL 2.0, not flat XES, as the target log format.** COMISET's 252,219,119 Windows events (202,304,794 from the real environment + 49,914,325 from the lab environment, per Table 1 of Pérez-Sánchez, Palacios & López López, *Data in Brief* 61:111723, 2025, DOI 10.1016/j.dib.2025.111723) naturally bind to multiple objects per event (host, process_guid, parent_guid, user) — exactly the convergence/divergence scenario OCEL 2.0 was designed to solve, with MITRE ATT&CK technique IDs as the natural activity label. [\(nih\)](#)
 - **Treat "event log construction" as the hardest, most consequential step.** Use the PM² methodology of van Eck, Lu, Leemans & van der Aalst (CAiSE 2015) as a backbone, and explicitly compare your four candidate case aggregations (host+time_window, host+process_guid, host+root_process_guid, host+episode) as *flattenings* of a single underlying OCEL — that way only one extraction pipeline is needed, and each "case notion" is just `pm4py.objects.ocel.util.flattening.flatten(ocel, object_type)`. [Readthedocs](#)
 - **Expect to engineer for scale and accept that you are doing original research.** No published paper to date constructs an OCEL 2.0 log from Sysmon/Windows telemetry; the closest prior art are Alvarenga et al. (*Computers & Security* 73, 2018), Rodríguez, Betarte & Calejari (LADC 2023 / JISA 2024) and event-knowledge-graph work by Fahland et al. (*Process Mining Handbook*, 2022). 252 M events exceeds pm4py's published comfort zone ($\approx$ tens of millions), so plan for SQLite-backed OCEL 2.0 storage and chunked construction.
-

Key Findings

1. **OCEL 2.0 is the right standard.** Released on **October 16, 2023** (specification version date in arXiv:2403.01975; Berti, Koren, Adams, Park, Knopp, Graves, Rafiei, Liß, Tacke Genannt Unterberg, Zhang, Schwanen, Pegoraro, van der Aalst), it extends OCEL 1.0 (2020) with **qualified event-to-object (E2O) and object-to-object (O2O) relationships, evolving object attribute values**, and three exchange formats: **SQLite (relational), XML, and JSON**. The SQLite format is the only one practical for 252 M events. [\(arXiv + 3\)](#)
2. **Case-notion selection is the crux of event-log construction.** Van der Aalst showed that forcing a single case ID causes **convergence** (one event duplicated across cases) and **divergence** (multiple instances of the same activity collapsed into one case). Adams, Schuster, Schmitz, Schuh & van der Aalst (ICPM 2022, arXiv:2208.03235) proved that *connected-component extraction* is the only case-notion strategy free of convergence, divergence, and deficiency. [\(ResearchGate +2\)](#) [\(ResearchGate\)](#)
3. **pm4py natively supports OCEL 2.0.** `pm4py.read.read_ocel2_sqlite/xml/json` and `pm4py.write.write_ocel2_sqlite/xml/json`, plus

`pm4py.convert.convert_log_to_ocel(df, activity_column, timestamp_column, object_types=[...])` and `pm4py.objects.ocel.util.extended_table.get_ocel_from_extended_table(df)` let you build an OCEL directly from a pandas DataFrame whose schema follows `ocel:eid, ocel:timestamp, ocel:activity, ocel:type:<OBJECTTYPE>`. Flattening to a traditional log for any single object type is one call:

`pm4py.objects.ocel.util.flattening.flatten(ocel, ot)`. (Processintelligence + 3)

4. **Cybersecurity-specific OCEL work is a gap.** Direct OCEL applications to Sysmon/IDS are essentially absent in the literature as of May 2026. The closest references are: Alvarenga, Barbon Jr., Miani, Cukier & Zarpelão (*Computers & Security* 73:474–491, 2018) on process mining + hierarchical clustering of IDS alerts; Rodríguez, Betarte & Calejari (LADC 2023, DOI 10.1145/3615366.3615372; extended in *J. Internet Services & Applications* 2024) mapping Sysmon events to MITRE ATT&CK techniques as PM activities; Macak, Daubner, Fani Sani & Buhnova (ADMA 2022, DOI 10.1007/978-3-030-95405-5_28) systematic review of PM in cybersecurity; and Fahland's event-knowledge-graph chapter in the *Process Mining Handbook* (LNBIP 448, 2022). (ScienceDirect) (Sbc)
 5. **Your four aggregation strategies map cleanly onto OCEL constructs**, eliminating the need to extract four separate logs. Each is a different *case extraction* over the same OCEL: host+process_guid is leading-type extraction on `process`; host+root_process_guid is the connected component of process-parent edges; host+time_window is a non-graph slicing; host+episode is a session/episode rule over per-host event sequences.
-

Details

Topic 1 — OCEL: What it is and why it solves your problem

The flat-log problem. Traditional process mining starts from an event log in which each event is bound to exactly one case. XES (IEEE Std 1849-2016) formalises this. The Process Mining Manifesto (van der Aalst et al., BPM Workshops 2011, LNBIP 99:169–194) lists "correlation" — assigning events to cases — as one of the fundamental challenges. When the underlying data has many-to-many relationships (and Sysmon data overwhelmingly does: one network-connection event belongs to a process AND a host AND a user AND a remote IP), forcing a single case ID causes:

- **Convergence:** the *same* event is duplicated across multiple cases. Example for Sysmon: if you choose `user_name` as the case, a single "process creation" event gets copied into every user-session case that touches that process, inflating frequency counts. (Springer) (Celonis)
- **Divergence:** multiple instances of the same activity collapse into one case, hiding the order in which they happened. Example: if you choose `host_name` as the case, every "DLL load" on a given machine appears as repeated activities in one giant trace with no separation between different attacker actions.
- **Deficiency:** events fit no case at all and are silently dropped.

Adams & van der Aalst (ICPM 2021, "Precision and Fitness in Object-Centric Process Mining," DOI 10.1109/ICPM53251.2021.9576886) gave the canonical example: a *Load cargo* event for both `plane1` and `bag1` has no single legitimate case notion. Sysmon has the same shape: a *Process Create* event simultaneously references the parent process, the child process, the user, and the host. [arxiv](#)

OCEL 1.0 (2020) → OCEL 2.0 (2023). OCEL 1.0 (Ghahfarokhi, Park, Berti, van der Aalst, BPM 2021, DOI 10.1007/978-3-030-85082-1_16) introduced a JSON/XML format in which an event can refer to *any number* of objects of any number of types, but had three deliberate limitations: no qualifier for relationships, no object-to-object relationships, and no time-varying object attributes. OCEL 2.0 (Berti et al., specification document dated October 16, 2023, published as arXiv:2403.01975 in March 2024) fixes all three and adds a relational SQLite format. The metamodel has the elements: `event type → event → event attribute / event attribute value`, `object type → object → object attribute / object attribute value (timestamped)`, plus qualified E2O and O2O relations. [Ocel-standard + 3](#)

Multi-object events. This is the heart of why OCEL fits cybersecurity. An OCEL event has no "case" field; instead it has a *list of E2O relationships*. A Sysmon EventID 1 (Process Create) becomes one OCEL event with E2O links to four objects: a `host` object, a `process` object (child), a `process` object (parent — via O2O `parent-of` link), and a `user` object — each link can be qualified ("created", "spawned-by", "as-user"). The MITRE ATT&CK technique ID, where labelled, becomes the **event type** (activity name).

Schema (JSON example, OCEL 2.0). From <https://www.ocel-standard.org/specification/formats/json/>, an OCEL contains `eventTypes`, `objectTypes`, `events` (each with `id`, `type`, `time`, `attributes`, `relationships[{objectId, qualifier}]`), and `objects` (each with `id`, `type`, `attributes[{name, time, value}]`), and `relationships[{objectId, qualifier}]` for O2O).

Tool support. Per the official pm4py page on ocel-standard.org: "pm4py ... has complete compatibility with the OCEL 2.0 specification. It can ingest, process, and store object-centric event logs in both the relational and XML formats defined in OCEL 2.0." Other tools: **ocpa** (Berti et al.) for discovery and predictive monitoring of OCELS; **ProM** ([OCELStandard](#) package) for visualization; **pm4js** for JavaScript environments. [Doks](#) [Springer](#)

Cybersecurity OCEL literature. The honest finding: there is no peer-reviewed paper that uses OCEL 2.0 for Sysmon/Windows telemetry. The closest are (i) Hamdi, Elleuch, Laga & Gaaloul, "Extracting Object-Centric Event Logs from Incident Data Using Large Language Models," CoopIS 2025 / LNCS 15535 (Springer 2026, DOI 10.1007/978-3-032-15538-2_4), which constructs OCELS from industrial incident data with multiple objects (hardware, software); and (ii) Fahland et al., "Process Mining over Multiple Behavioral Dimensions with Event Knowledge Graphs," in the *Process Mining Handbook*, LNBIP 448, Chapter 9 (Springer 2022, DOI 10.1007/978-3-031-08848-3_9), which is the EKG counterpart to OCEL and explicitly addresses the multi-entity problem. [Springer](#)

Topic 2 — The event-log construction phase

The **PM² methodology** (van Eck, Lu, Leemans & van der Aalst, CAiSE 2015, LNCS 9097:297–313, DOI 10.1007/978-3-319-19069-3_19) defines six stages: (1) Planning, (2) Extraction, (3) Data Processing, (4) Mining & Analysis, (5) Evaluation, (6) Process Improvement & Support. The extraction stage explicitly produces *event data*, which becomes an *event log* only after a case notion and event-class (activity) definition are imposed in the data-processing stage. (QUT ePrints)

(Leemans)

Three key decisions identified by both PM² and the Process Mining Manifesto:

1. **Case notion (the hardest).** This is the equivalence relation that partitions events into traces. Van der Aalst calls choosing it the central challenge because **the case notion is a modelling choice, not a property of the data**. Different case notions yield different (and equally legitimate) views of the same process.
2. **Activity definition (event-class).** What raw field becomes the activity label? In COMISET, the MITRE ATT&CK technique ID (`T1059.001`, `T1003`, etc.) is the obvious choice for labelled events; for unlabelled events, you may need to abstract Sysmon EventID + ImageName or use a hierarchical abstraction.
3. **Timestamp selection.** Atomic vs. interval; transactional lifecycle (start/complete); clock-skew across hosts. Rodríguez, Betarte & Calejari (LADC 2023) explicitly use the Sysmon `UtcTime` field as the event timestamp.

Is it art or science? Both. The Process Mining Manifesto (Guiding Principle GP1 — "Event Data Should Be Treated as First-Class Citizens" — and Challenge C4 — "Dealing with Concept Drift") frames it as engineering judgement constrained by data-quality criteria (the manifesto's five-star event-log maturity scale). PM² gives a methodology. But the actual *choice* of case notion remains craft. Andrews et al. ("Quality-informed semi-automated event log generation for process mining," *Decision Support Systems*, 2022) is one of the few systematic frameworks. An object-centric methodological extension exists: Berti, Park et al., "OCPM²: Extending the Process Mining Methodology for Object-Centric Event Data Extraction" (arXiv:2503.10735, 2025).

(Springer + 2)

Convergence and divergence (formal). Van der Aalst & Berti, "Object-Centric Process Mining: Dealing with Divergence and Convergence in Event Data" (SEFM 2019, LNCS 11724:3-25, DOI 10.1007/978-3-030-30446-1_1) is the canonical statement. The Celonis explanation captures it: "Convergence happens when one event is replicated across different cases ... Divergence happens when there may be multiple instances of the same activity within a single case." Van Detten et al., "A Framework for Advanced Case Notions in Object-Centric Process Mining" (ICPM Workshops 2024) adds **deficiency** (events with no case at all). (Celonis + 2)

The strongest formal result, which is directly applicable to COMISET: Adams et al. (ICPM 2022, arXiv:2208.03235) proved that the **connected-component case notion** on the event-object graph is the only extraction strategy guaranteed free of all three pathologies, but warned that connected components can grow to encompass the entire log if the data is highly connected — which it definitely is for a long-running host. (Springer)

Topic 3 — Cybersecurity event-log construction

Alvarenga et al. (2018). Their approach in *Computers & Security* 73:474-491 takes IDS alerts (Snort), defines a case as a *correlated alert group* (IP-based correlation), uses attribute matching plus hierarchical clustering to group alerts into attack scenarios, then applies process discovery to visualize the attacker's strategy. The case notion is therefore *the attack scenario as discovered by clustering* — not an a priori field — which is the standard pattern in security PM but discards information by forcing a single ID. (ScienceDirect) (ResearchGate)

Rodríguez, Betarte & Calegari (2021, 2023, 2024). Their 2021 IEEE URUCON paper introduced process-mining-based attacker profiling. The 2023 LADC paper (DOI 10.1145/3615366.3615372) is the most relevant to COMISET: they take Sysmon EVTX logs, run them through Zircolite (SIGMA-based) to label events with MITRE ATT&CK techniques/tactics, then build event logs where **activity = MITRE technique** and **case = the per-host event session/cluster from the dataset's own grouping**, with the Sysmon timestamp as event time. The 2024 *J. Internet Services & Applications* extension uses Sysmon EventID 12 (registry changes) and EventID 2 (file timestamp changes) as additional indicators. Crucially, they don't use OCEL — they flatten to a per-host case notion and accept the resulting divergence. (Semantic Scholar + 2)

Other cybersecurity PM work: Myers, Radke, Suriadi & Foo (ICT Systems Security and Privacy Protection, 2017) for ICS attack detection; Zhong, Goulermas & Lisitsa (arXiv:2205.12064, 2022) for online IDS preprocessing; Bahrani & Bidgly (ISC 2019) for ransomware detection with PM + classification; Macak, Daubner, Fani Sani & Buhnova systematic review (ADMA 2022) covering 2014-2020. (Springer)

Why OCEL fits security better than a single case ID. In Sysmon data, a *Process Create* event (EventID 1) by definition involves five entities: the host, the new process, its parent process, the user account, and (often) the originating logon session. The threat-hunter playbook (https://threathunterplaybook.com/pre-hunt/data_modeling.html) explicitly notes: "We can model Sysmon events and find that almost all its events can be correlated by the ProcessGUID field ... We can then use that model and start mapping events to adversarial activity. For example, we can map Windows Management Instrumentation (WMI) spawning a new process that makes a network connection to communicate with an external network entity to specific Sysmon event logs joined by their ProcessGUID value." OCEL formalizes exactly this: ProcessGUID and ParentProcessGUID become **objects** of type (process), the parent-child relationship becomes a qualified **O2O** edge, and each event has E2O links to all participating objects without any flattening loss. (Threathunterplaybook)

Topic 4 — Practical guidance for COMISET

Recommended OCEL schema for COMISET:

OCEL element	Source field in COMISET
<code>event:id</code>	unique row id
<code>event:timestamp</code>	Sysmon UtcTime
<code>event:type</code> (activity)	MITRE ATT&CK technique ID for labelled events; else Sysmon EventID + ImageBaseName
Event attributes	EventID, CommandLine, Hashes, integrity level, network 5-tuple, etc.
Object type <code>host</code>	host_name (anonymized like <code>computer01</code>) <code>(nih)</code>
Object type <code>process</code>	process_guid; attribute: ImageName, CommandLine, IntegrityLevel
Object type <code>user</code>	user_name (anonymized like <code>sxxc</code>) <code>(nih)</code>
Object type <code>file</code>	TargetFilename / ImageLoaded (for EID 11, 7)
Object type <code>registry_key</code>	TargetObject (for EID 12, 13, 14)
Object type <code>network_endpoint</code>	DestinationIp:DestinationPort (for EID 3)
O2O qualifier <code>parent-of</code>	process_guid $\leftrightarrow$ parent_guid
O2O qualifier <code>runs-on</code>	process $\leftrightarrow$ host
O2O qualifier <code>executes-as</code>	process $\leftrightarrow$ user

This schema captures the full graph without forcing any single case ID.

pm4py construction code (the three viable paths):

```
python
```

```

import pm4py
import pandas as pd

# ---- PATH A: from a flat DataFrame using convert_log_to_ocel ----
# (works when columns hold single object IDs, not lists)
df = pd.read_parquet("comiset_clean.parquet")
df = df.rename(columns={"UtcTime": "time:timestamp",
                        "technique_id": "concept:name"})
ocel = pm4py.convert.convert_log_to_ocel(
    df,
    activity_column="concept:name",
    timestamp_column="time:timestamp",
    object_types=["host_name", "process_guid",
                  "parent_guid", "user_name"],
    additional_event_attributes=["EventID", "CommandLine"]
)

# ---- PATH B: from an "extended table" with list-typed object columns ----
# columns: ocel:eid, ocel:timestamp, ocel:activity,
#          ocel:type:host, ocel:type:process, ocel:type:user, ...
from pm4py.objects.ocel.util.extended_table import get_ocel_from_extended_table
ocel = get_ocel_from_extended_table(df_extended)

# ---- PATH C: construct OCEL directly from 3 DataFrames (most scalable) ----
from pm4py.objects.ocel.obj import OCEL
ocel = OCEL(events=events_df, objects=objects_df, relations=relations_df)

# Write OCEL 2.0 SQLite (the only format suitable for 252 M events)
pm4py.write.write_ocel2_sqlite(ocel, "comiset.sqlite")

# Flatten to a traditional log on a chosen object type
from pm4py.objects.ocel.util.flattening import flatten
trad_log = flatten(ocel, "process_guid") # one trace per process

```

The `convert_log_to_ocel` signature is documented at the pm4py 2.7.8 API:

```

pm4py.convert.convert_log_to_ocel(log, activity_column='concept:name',
timestamp_column='time:timestamp', object_types=None, obj_separator=' AND ',
additional_event_attributes=None) → OCEL. (Fraunhofer)

```

Your four aggregation strategies, re-cast as OCEL operations.

Your strategy	OCEL implementation
host + time_window	A non-graph slicing: bucket events by <code>host</code> object $\times$ time window, treat each bucket as a case. In OCEL, achieved by (a) flattening on <code>host</code> , then (b) splitting the per-host trace at gap $> \Delta$. Conceptually simplest, most prone to divergence (many techniques can run in a window) but no convergence.
host + process_guid	Leading-type extraction on <code>process</code> (Adams et al. ICPM 2022). <code>arXiv</code> Each case = the events of one process plus its directly co-occurring host/user. Equivalent to <code>flatten(ocel, "process_guid")</code> . <code>Readthedocs</code> Excellent control flow, but loses cross-process attack chains.
host + root_process_guid	A bigger leading-type extraction: case = all processes in the descendant tree of one root, derived by transitive closure of the O2O <code>parent-of</code> edge. Captures full ancestry of multi-stage attacks (initial cmd.exe $\rightarrow$ powershell $\rightarrow$ mimikatz). Best balance of fidelity and case size for APT analysis.
host + episode	A connected-component extraction (Adams et al. 2022, van Detten et al. 2024) <code>Vdaalst</code> restricted to per-host sub-graphs and segmented by activity-rate or session-idle rules (cf. Toivonen & Mannila's frequent-episode framework; Sadoddin & Ghorbani's RTECA, <i>Computers & Security</i> 2014 — DOI 10.1016/j.cose.2014.10.006). Most theoretically sound (free of convergence/divergence/deficiency), but episode boundary rules need tuning.

The decisive insight: **build the OCEL once, then derive all four flattenings programmatically**. This is the proper way to *compare* the strategies in your study, because all are operating on the same underlying event-object graph.

Predictive Process Monitoring on OCELS. Recent OCPM PPM approaches include: HOEG (Smit, Reijers & Lu, arXiv:2404.05316, 2024) — heterogeneous object-event graphs with GNNs; Gherissi, Acheli, El Haddad & Grigori, "Predictive Process Monitoring Using Object-Centric Graph Embeddings," ICSOC 2024 Workshops, LNCS 15833 (Springer Singapore, 2026, DOI 10.1007/978-981-96-7238-7_5) — graph attention + LSTM for next-activity / next-event-time on OCELS; Adams et al.'s feature-extraction framework; ocpa library for object-centric prediction. For COMISET, predicting *the next MITRE technique in an attack chain* is the natural task, and HOEG-style heterogeneous graph encoders are the current SOTA for keeping object diversity.

`arxiv + 2`

Scalability warning for 252 M events

pm4py keeps `ocel.events`, `ocel.objects`, `ocel.relations` as in-memory pandas DataFrames. The published OCPM ceiling on commodity hardware is "tens of millions" of events. Bosmans, Peeperkorn, Goossens, Lugaresi, De Smedt & De Weerd (KU Leuven), "Dynamic and Scalable Data Preparation for Object-Centric Process Mining" (arXiv:2410.00596, October 2024) propose "a database format designed for an intermediate data storage hub, which

segregates process mining applications from their data sources using a hub-and-spoke architecture" — the most directly relevant scalable-storage work for the 250 M-event range. For 252 M events you should: [arxiv](#)

- Persist the OCEL in **SQLite** (OCEL 2.0 relational format) from the start. The OCEL 2.0 spec recommends it: "OCEL 2.0 offers three exchange formats: a relational database (SQLite), XML, and JSON format." [arXiv](#) [arXiv](#)
 - Build the events/objects/relations DataFrames in chunks, write to SQLite, never hold the full graph in RAM.
 - Run discovery/PPM on **filtered sub-OCELS** — e.g., filter by labelled-malicious + a sliding context, or by attack-stage (initial access, execution, persistence ...) using `pm4py.filtering.filter_ocel_*` functions.
 - For deep learning PPM, sample process executions rather than flatten the whole log.
-

Recommendations

Staged plan

Stage 0 — Reproduce the literature baseline (1-2 weeks). Implement Rodríguez et al.'s pipeline as a sanity check: flat XES, case = per-host session, activity = MITRE technique, run Inductive Miner in pm4py. This gives you a calibrated baseline and a sense of divergence/convergence pain.

Stage 1 — Build the canonical OCEL 2.0 log (3-6 weeks).

- Define the object schema in the table above.
- Stream COMISSET's JSON files through a Spark/Polars pipeline that emits three Parquet tables (events, objects, relations) conforming to OCEL 2.0.
- Use `pm4py.objects.ocel.obj.OCEL(events_df, objects_df, relations_df)` plus `pm4py.write.write_ocel2_sqlite(...)` to materialise an OCEL 2.0 SQLite database.
- Validate against <https://www.ocel-standard.org/2.0/ocel20-schema-relational.pdf>.

Stage 2 — Derive your four flattenings programmatically (1 week). Implement each of host+time_window, host+process_guid, host+root_process_guid, host+episode as a Python function that takes the OCEL and returns a traditional `EventLog`. This is your "case-notion ablation study."

Stage 3 — Run process discovery and PPM on each flattening (4-8 weeks). Use Inductive Miner / Split Miner for discovery; for PPM, benchmark a flat LSTM, ProcessTransformer (Bukhsh, Saeed & Dijkman, arXiv:2104.00721), and an OCEL-native model (HOEG, arXiv:2404.05316) on next-technique prediction.

Stage 4 — Publish the gap-filling contribution. Frame the paper as "OCEL 2.0 for endpoint security telemetry": the first OCEL construction methodology for Sysmon-class logs, with

empirical comparison of four case notions on a 252 M-event labelled dataset.

Decision thresholds that should change the plan

- If OCEL 2.0 SQLite construction is too slow or storage > 200 GB → fall back to OCEL 1.0 JSON sharded per host and merge analytically; consider EKG (Neo4j) representation as in Fahland et al. 2022.
 - If connected components in the host+episode strategy are degenerate (one huge component per host) → switch to leading-type extraction on `process` (host+process_guid) or apply van Detten et al.'s correctness/simplicity optimization (ICPM Workshops 2024).
 - If PPM models with object-centric features do not beat flattened baselines by a meaningful margin → report it as a negative result; Fioretto & Masciari's comparative review ("A comparative analysis of predictive process monitoring: object-centric versus classical event logs," *Knowledge and Information Systems* 67(9):7355-7398, May 25, 2025, DOI 10.1007/s10115-025-02461-y) surveys multiple studies — notably Galanti & de Leoni (BPM 2023 Workshops, DOI 10.1007/978-3-031-50974-2_39) reported CatBoost on engineered features outperforming GNNs in training time and accuracy — so a negative result is plausible. [ResearchGate](#) [Springer](#)
-

Caveats

1. **Open research area.** No published peer-reviewed paper constructs OCEL 2.0 from Sysmon/Windows telemetry. You are doing original methodological work; expect that "best practice" must be partly invented and partly transferred from supply-chain/healthcare OCEL applications.
2. **Scale beyond pm4py's comfort zone.** 252 M events is roughly one order of magnitude larger than the OCPM datasets in the literature. Plan for out-of-core construction and don't expect interactive visualization of the full log.
3. **MITRE ATT&CK label quality.** The Zircolite/SIGMA labelling pipeline used by COMISET inherits all the false-positive/false-negative properties of SIGMA rules; some events labelled as a single technique may legitimately belong to multiple, and unlabelled-but-malicious events exist (COMISET reports 631,606 malicious events in the real environment, 1,713,709 in the lab environment, against ~252 M total — labels are sparse). [nih](#)
4. **Connected components can explode.** Adams et al. (ICPM 2022) warn that on highly-connected event-object graphs (a busy Windows host certainly qualifies), the connected-component case notion can yield a single giant case per host per day. Episode boundaries are an essential cutoff and need tuning. [Springer](#)
5. **PM4py API churn.** The exact OCEL function names (`read_ocel2_sqlite`, `write_ocel2_sqlite`, etc.) appear in pm4py ≥ 2.7. Pin a version in your environment.
6. **Activity-label granularity vs. trace length.** Using MITRE technique IDs (~200 leaves on Enterprise matrix) gives short, interpretable traces; using raw Sysmon EventID + Image

gives much longer, noisier traces. You may need a hybrid (technique where labelled, technique-equivalent abstraction elsewhere).

7. **Clock skew across hosts.** Sysmon UtcTime is per-host; cross-host correlation (lateral movement) requires NTP-quality timestamps or explicit network-event linking. Don't trust microsecond-level cross-host ordering.